

FACULTY OF COMPUTER SCIENCE

CSC 410: Distributed Systems

Lecture 1: Introduction

Dr. Hossam AbdelRahman

What is DS?

- **What?**
  - Multiple computers communicating via a network…
  - Trying to achieve some task together
  - Consists of “nodes” (computer, phone, car, robot, …)
- **Why?**
  - Scalable web services
  - Big data processing and management
  - The large-scale infrastructure systems making these possible

Importance of DS

- **It's inherently distributed:**
  - e.g. sending a message from your mobile phone to your friend's phone
- **For better reliability:**
  - even if one node fails, the system as a whole keeps functioning
- **For better performance:**
  - get data from a nearby node rather than one halfway round the world
- **To solve bigger problems:**
  - e.g. huge amounts of data, can't t on one machine

List of References

- Lecture Notes
- Lab Documents
- Introduction to Reliable and Secure Distributed Programming (2nd ed), Cachin, Guerraoui & Rodrigues, Springer 2011
- Distributed Systems: Principles and Paradigms, Andrew Tanenbaum and Maarten van Steen, Prentice Hall.
- Principles of Computer System Design. Jerome Saltzer and M. Frans Kaashoek, Morgan Kaufmann.

Learning Objective

- **1. Distributed systems primer**: challenges and goals of distributed systems.
- **2. Communication models**: remote procedure calls (RPC), RPC libraries, failure models, semantics.
- **3. Time and coordination**: challenges, physical and logical clocks, distributed mutual exclusion.
- **4. Sharding, replication, and the agreement problem**: commitment and consensus, use cases for each.
- **5. Local transactions (background)**: ACID semantics, concurrency control, recovery mechanisms.
- **6. Commitment protocols:** 2-phase commit, 2-phase commit, safety/liveness tradeoffs with the two.
- **7. Consensus protocols**: Paxos overview, key ideas, basic algorithm, examples, liveness failure mode.
- **8. Replication architectures:** fault-tolerant architectures (primary/secondary, master/slaves); the design of Google's
- Chubby lock service; the 2PC+Paxos approach to both scalability and fault tolerance.
- **9. Case study: Google's Spanner:** TrueTime, Spanner and its fault-tolerant, linearizable, distributed transactions.
- **10. Other consistency and isolation semantics**: sequential, causal, and eventual consistency; a few isolation semantics;
- mechanisms to achieve each; tradeoffs between them.
- **11. Distributed computation:** MapReduce design, TensorFlow design, approximate computing engines.
- **12. Distributed systems security primer**: authentication protocols, Needham-Schroeder, Kerberos, byzantine fault
- tolerance, maybe blockchain.

Course Assessments

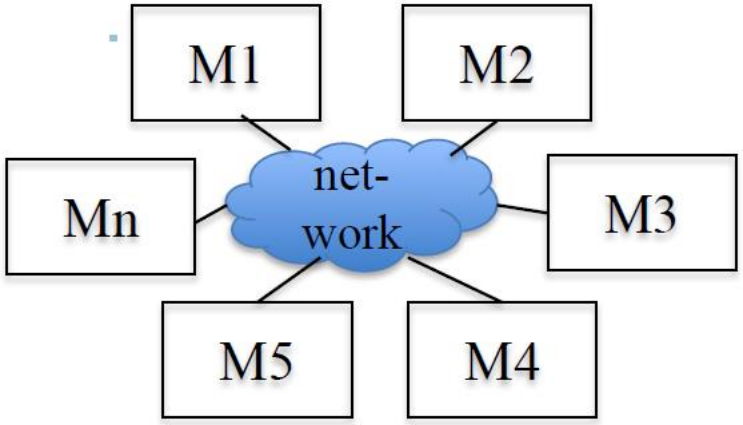
- Midterm Exam 30
- Project 10
- Submission 5
- Quizzes 10
- Participation 5
- Final Exam 40

Distributed Systems

- Distribution is hard, for many reasons.
- Distributed Systems (DS) aim to provide the core mechanisms that address the challenges and hide them under abstractions.
- Not all challenges can be hidden, so DS developers and users must understand the challenges and approaches to address them.
- Next we'll talk about some of the **challenges**, **goals**, and **approaches** in DS design**.**

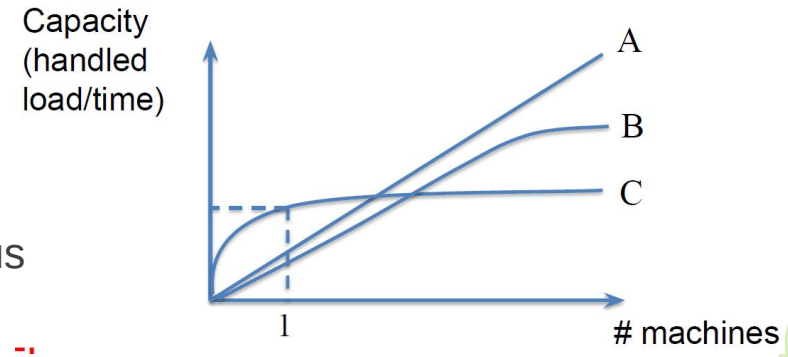
Challenge 1: Ever-Growing Load

- Data is big. Users are many. Requests are even more. No single machine can handle the ever-growing load.
- A primary goal of DSes is to enable *"multiple independent machines interconnected through a network to coordinate to achieve a common, consistent service or function."*
- But effective coordination at scale is hard!



Goal 1: Scalability

- The more resource you add, the more data you should be able to store/process, and the more users you should be able to serve.
- But never hope for **perfect scalability**: add one machine, increase your capacity proportionally forever.
  - Most often, the scalability curve tapers off as various components of the system start to reach their *capacities* (maximum handled load/unit of time, e.g., max number of requests completed per second).
  - Components that limit scalability are called *bottlenecks* and sometimes they can be very hidden (logging server, network router).



Approach 1: Sharding

- Launch multiple processes (a.k.a., *workers*), split up your load into pieces (a.k.a., *shards*), and assign different shards to different workers.
- The workers should be designed to coordinate to achieve a common, consistent service despite the sharding.
- Unfortunately, sharding raises semantic challenges esp. with failures.

Challenge 2: Failures

- Many types of failures:
  - network, machine (software, hardware, flipped bits in memory,..), datacenter,…
  - some are small and isolated, others are major failures
  - some are persistent, others are temporary
  - some resolve themselves, others require human intervention …
- At scale, failures are **almost guaranteed to occur all the time**, and most of them occur at unpredictable times.
- The big challenge is that failures greatly challenge coordination between the machines of a distributed systems.

Goal 2: Fault Tolerance

- Goal is to hide failures as much as possible to provide a service that for example, finishes the computation fast despite failures, stores some data reliably despite failures, …
- Fault tolerance subsumes:
  - **Availability:** the service/data continues to be operational despite failures.
  - **Durability:** some data or updates that have been acknowledged by the system will persist despite failures and will eventually become available.
- Durability differs from availability: durability can be thought of as "eventual availability" of some data or state.

Approach 2: Replication

- Have multiple replicas execute/store the same shard; if one replica dies, another replica can provide the data/computation.
- Example:
  - Say you're computing a maximum over a sharded dataset across multiple workers.
  - Have the maximum over each shard be computed by 2-3 replicas in parallel.
  - If one replica dies, another one can report the max to the other n-1 worker sets.

Challenge 3: Consistency in Sharded and Replicated Systems is HARD

- Example:
  - Say we want to compute an exact average over a sharded and replicated dataset.
  - How do we make sure that we incorporate the average over each shard only once if the average over each shard is computed and reported by three replicas?
  - Assigning each shard a unique ID may help, but what if the faulty replica does not die, but instead spews up a faulty value due to, e.g., memory error.

Goal 3: Meaningful Consistency Semantics

- Dealing with these issues is hard for both the programmers and users.
- The goal is to build infrastructure systems (such as storage systems, computation frameworks, etc.) that provide **clear semantics** that hold in the face of failures, and to express those semantics clearly in APIs.
- Example:
  - Say you decide that in case of failure your distributed computation system will return the results of a partial computation.
  - Then you need to communicate that through your API. You should also provide error bounds for the results you are reporting. That's *semantic*.

Approach 3:Rigorous Protocols

- The general approach is to develop rigorous protocols, which we will generically call *agreement protocols*, that allow workers and replicas to coordinate in a consistent way despite failures.
- Agreement protocols often rely on the notion of *majorities*: as long as a majority agrees on a value, the idea is that it can be safe to continue making that action.
- Different protocols exist for different consistency challenges, and often the protocols can be composed to address bigger, more realistic challenges.

Summary of Challenges & Goals

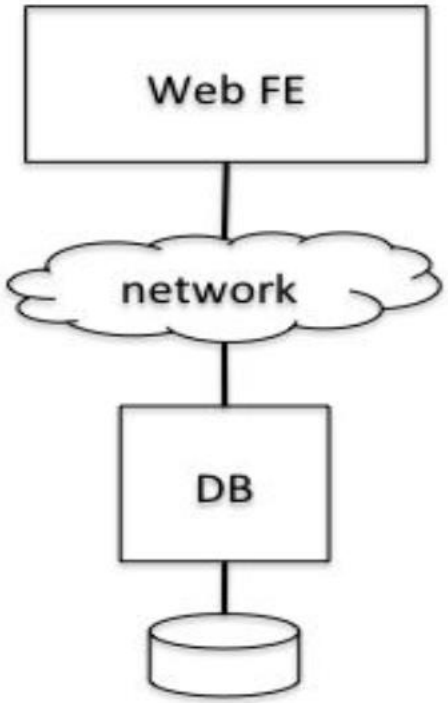
- **The trouble with distributed systems:**
  - Communication may fail (and we might not even know it has failed).
  - Processes may crash (and we might not know).
  - All of this may happen non-deterministically.
- **Fault tolerance:** we want the system as a whole to continue working, even when some parts are faulty.
- This is hard.
- Writing a program to run on a single computer is comparatively easy

The Aim of the Course

- Scalability, fault tolerance, consistency, and performance are difficult goals to achieve together.
- Solving them requires rigorous protocols and system architectures.
- Here, we will learn such protocols and architectures, plus how they are incorporated in real systems.

Example: Web Service Architecture
Basic Architecture

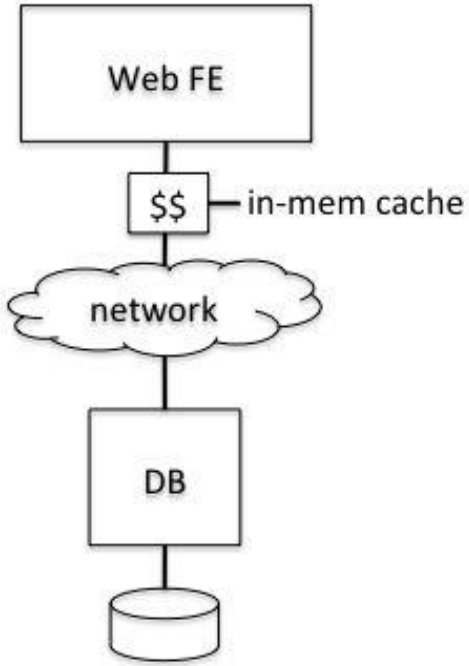
- Performance: poor
- Fault tolerance: poor
- Scalability: poor
- Semantics (consistency): Perfect



- **First, let's improve performance (latency).**

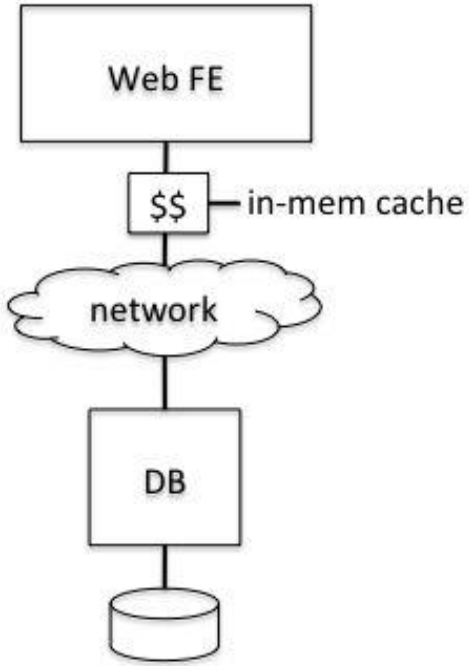
Goal: Reduce Latency

- Web FE's reads/writes go through in-memory cache ($$).
- Performance?
  - For reads?
  - For writes?
- Fault tolerance?
  - Availability?
  - Durability?
- Scalability?
- Semantics/consistency?



Goal: Reduce Latency

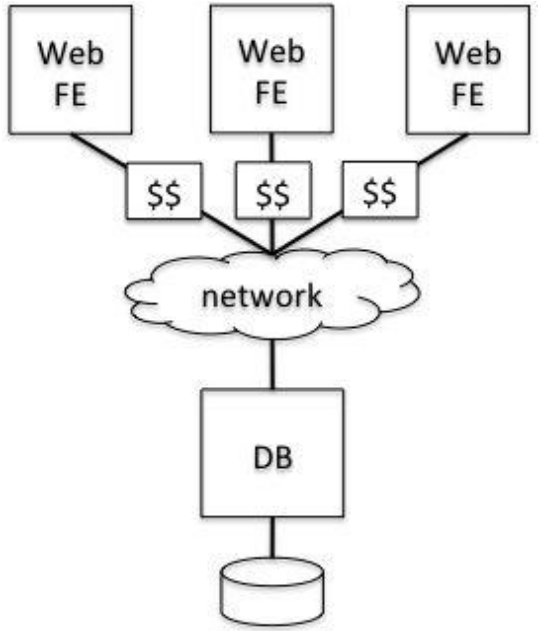
- Read latency: improved if working set fits in memory.
- Durability: depends on cache: good for *write-through* $$, poor for *write-back* $$.
- Write latency is opposite: good with write-back, poor with write-through.
- Consistency: good: you have 1 FE accesses DB, going through 1 $$, so behaviour is equivalent to single machine.



- **Let's deal with scalability, first on FE and later on DB.**

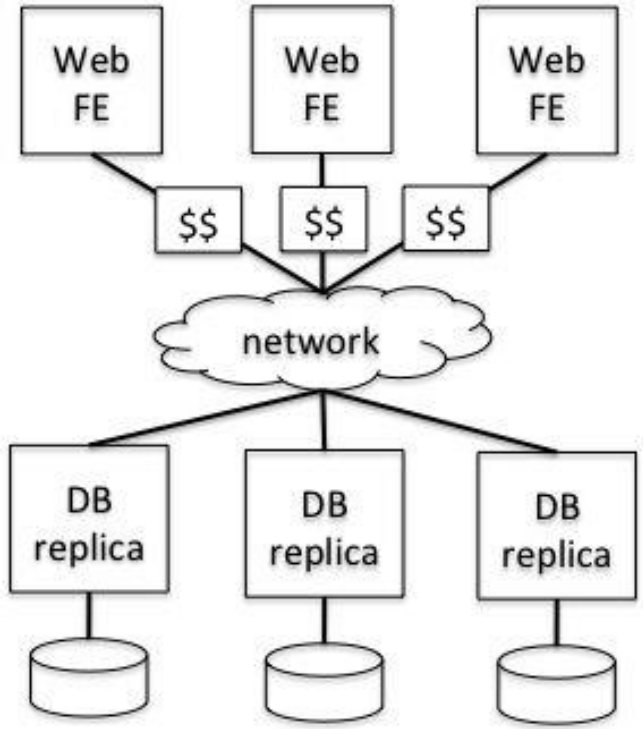
Goal: Scale Out the FE (and get fault tolerance for it)

- Launch multiple FEs. Each has its own local cache, which we'll assume is write-through.
  - Performance?
  - Fault tolerance?
  - Scalability?
  - Semantics (consistency)?



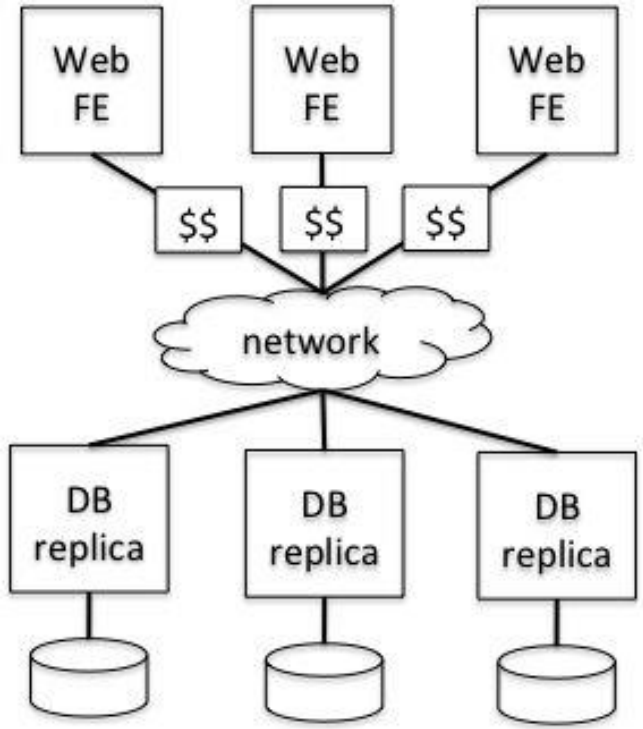
Goal: Fault Tolerance for DB

- Launch identical replicas of the DB server, each with its disk. All replicas hold all data, writes go to all.
  - Performance?
  - Fault tolerance?
  - Scalability?
  - Semantics (consistency)?



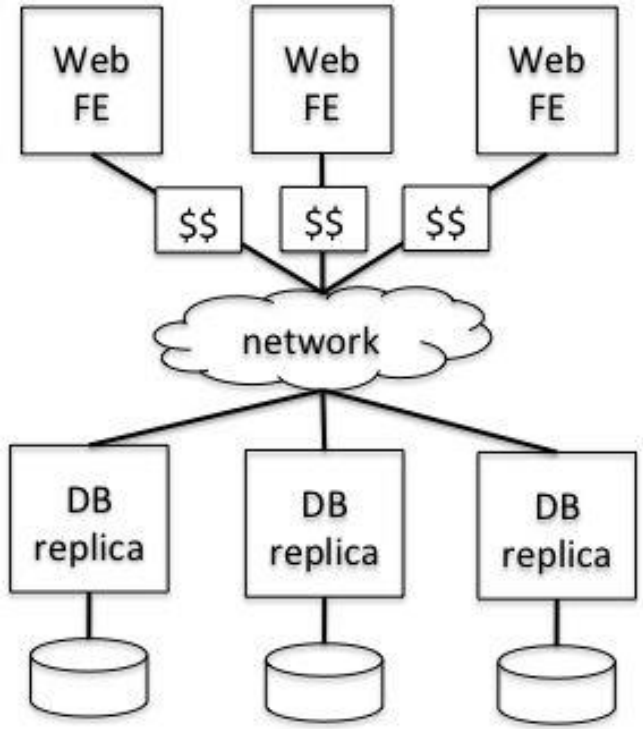
Goal: Fault Tolerance for DB

- Writes now need to propagate to all replicas. So they are much slower! Even if done in parallel, because FE now needs to wait for the slowest of DB replicas to commit (assuming write-through cache, which offers the best durability).
- All replicas must see all writes IN THE SAME ORDER! If order is exchanged, they can quickly go “out of sync”! So lots more consistency issues.



Goal: Fault Tolerance for DB

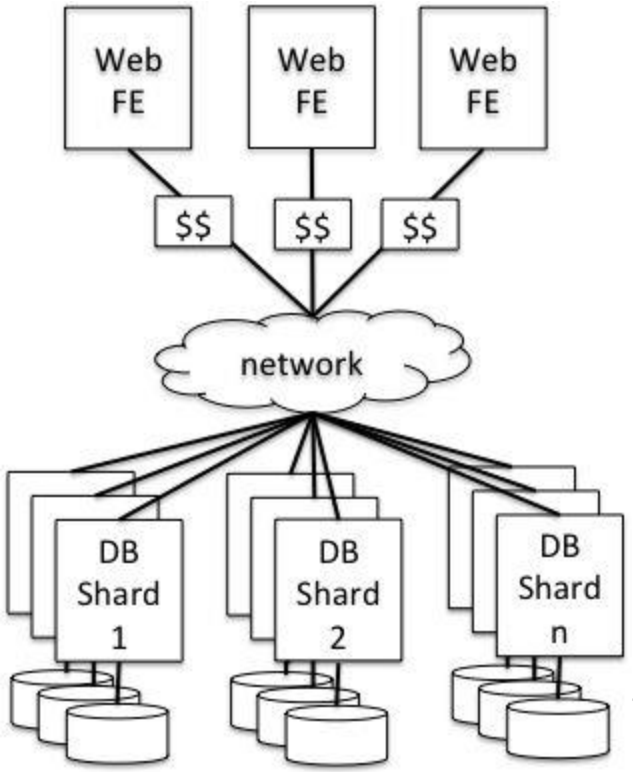
- There are also availability issues. If you require all the replicas to be available when a write is satisfied (for durability), availability goes DOWN! Consensus protocols, which work on a majority of replicas, address this.
- Another consistency challenge: how are reads handled? If you read from one replica, which one should you read given that updates to the item you're interested in might be in flight as you attempt to read it? We'll address these issues in future lectures by structuring the replica set.



- **Let's deal with DB scalability.**

Goal: Scale Out the DB

- Partition the database into multiple shards, replicate each multiple times for fault tolerance. Requests for different shards go to different replica groups.
  - Performance?
  - Fault tolerance?
  - Scalability?
  - Semantics (consistency)?



Goal: Scale Out the DB

- New challenges that arise:
  - How should data be sharded? Based on users, on some property of the data?
  - How should different partitions get assigned to replica groups? How do clients know which servers serve/store which shards?
  - If the FE wants to write/read multiple entries in the DB, how can it do that atomically if they span multiple shards? If different replica groups need to coordinate to implement atomic updates across shards, won't that hinder scalability?

